

1. Tutorial básico: tablas, expresiones y resaltado

Este capítulo presenta los elementos fundamentales del DSL mediante un ejemplo mínimo que iremos construyendo en tres pasos:

1. Crear una tabla vacía con solo los encabezados y exportarla (al backend Excel, por ejemplo) para verificar que el sistema funciona.
2. Añadir los datos de entrada.
3. Incorporar una columna calculada y resaltado condicional.

Al terminar sabrá cómo se definen las columnas de una tabla, cómo se añaden expresiones que se calculan solas, cómo se resaltan celdas según una condición y cómo se genera el archivo de salida.

1.1 Qué vamos a construir

Queremos una tabla con cuatro columnas:

Nombre	Nota 1	Nota 2	Promedio
Ana	85	90	87.5
Luis	70	75	72.5
Elena	92	88	90.0
Carlos	60	65	62.5
Sofía	78	82	80.0
Pedro	95	91	93.0

Además:

- La columna **Promedio** no se escribe a mano: se calcula automáticamente como el promedio de las dos notas.
- Hay un **número fijo** (80, por ejemplo) almacenado en el libro.
- Los nombres de las personas cuyo promedio supera ese número aparecen **resaltados en rojo**.

1.2 Conceptos básicos

Antes de escribir código, conviene entender la arquitectura del DSL:

- Una **tabla** es una estructura con columnas. Cada columna tiene un nombre interno (símbolo del DSL que se usa en las expresiones del lenguaje) y una etiqueta (texto visible del encabezado). La tabla se define con `def-table`.
- Una **hoja** es una página del libro que agrupa una o más tablas. Se crea con `hoja`.
- Un **libro** es el contenedor principal que reúne una o más hojas. Se define con `libro`.
- El libro se **exporta** a un backend concreto (por ejemplo, el backend Excel que acompaña al DSL) mediante `xl-generate` y `xl-run-generated`.

Todo el código se escribe en Common Lisp, en un archivo con extensión `.lisp`. El DSL se encarga de traducir esa descripción a un programa Python que, a través del backend Excel, genera el archivo de salida.

Para seguir este tutorial cree el archivo `tutorial-basico.lisp` dentro de la carpeta `tutorial-basico/` (donde ya están `dsl-directo.lisp` y el resto de los archivos del proyecto).

Nota: a lo largo del tutorial iremos completando el mismo archivo `tutorial-basico.lisp`. Al final tendrá el programa completo.

1.3 Paso 1: Tabla con solo los encabezados

Vamos a empezar por lo más básico: definir la estructura de la tabla y crear un libro que la contenga, sin datos todavía. El objetivo es confirmar que el pipeline completo funciona (definir, generar, exportar) y ver cómo se ven los encabezados en el archivo de salida.

1.3.1 Definir la tabla

La macro `def-table` define una tabla. Le damos un nombre y la lista de columnas. Cada columna se escribe como un par (`identificador "Etiqueta"`), donde `identificador` es un símbolo de Common Lisp con el que el DSL reconoce la columna (se usa en expresiones como `(col identificador)`

para referirse a ella) y "Etiqueta" es el texto que se mostrará como encabezado de columna en el archivo de salida. Ambos pueden ser distintos: el identificador es un símbolo interno del lenguaje, la etiqueta es externa (lo que ve quien abre el archivo).

Listing 1.1: def-table con etiquetas visibles.

```
1 (def-table tabla-notas
2   ((nombre "Nombre") (nota1 "Nota 1") (nota2 "Nota
3     2") (promedio "Promedio")))
4   :cell-height 1
   :cell-width 1)
```

Aquí:

- `nombre` es el identificador interno de la primera columna y "Nombre" es la etiqueta que aparecerá como encabezado de columna en el archivo de salida.
- `nota1` y `nota2` son las columnas de las notas.
- `promedio` es la columna donde más adelante pondremos la expresión.

La tabla anterior produce un encabezado como este en el archivo de salida:

Nombre	Nota 1	Nota 2	Promedio
--------	--------	--------	----------

(Por ahora no hay filas de datos; las añadiremos en el paso 2.)

Identificador vs. etiqueta

El DSL maneja dos nombres por columna:

- El **identificador** (p. ej. `nombre`) es un símbolo de Common Lisp que usa el lenguaje en las expresiones como `(col nombre)`. Es el que escribe quien programa el DSL.
- La **etiqueta** (p. ej. "Nombre completo") es el texto que aparece en el encabezado de la columna en el archivo de salida. Es lo que ve quien abre el archivo.

Pueden ser distintos. En el ejemplo, `nombre` es el identificador interno y "Nombre" (o "Nombre completo") es la etiqueta visible.

No ejecute este fragmento solo

La macro `def-table` solo tiene sentido dentro del DSL completo. Si intenta cargar este fragmento en SBCL sin haber cargado antes

`dsl-directo.lisp` obtendrá un error. No se preocupe: el programa completo que verá al final incluye la instrucción `(load ...)` necesaria.

1.3.2 Crear el libro y exportar

Con la tabla definida, creamos un libro con una hoja que contenga la tabla. Como aún no tenemos datos, el `tabla` se escribe sin `:data`.

Listing 1.2: Paso 1: libro con tabla vacía.

```
1 (def-table tabla-notas
2   ((nombre "Nombre") (nota1 "Nota 1") (nota2 "Nota
3     2") (promedio "Promedio")))
4   :cell-height 1
5   :cell-width 1)
6 (libro tutorial-basico
7   :hojas (list
8     (hoja "Notas"
9       (tabla tabla-notas)))
10 )
11 (xl-generate tutorial-basico "tutorial-basico.py")
12 (xl-run-generated "tutorial-basico.py")
```

Ejecute desde la terminal:

```
cd tutorial-basico/
sbcl --script tutorial-basico.lisp
```

Esto carga el DSL, construye el libro, genera `tutorial-basico.py` y lo ejecuta para crear `Archivo-Excel.xlsx` (a través del backend Excel). Al abrir el archivo en un programa de planilla verá una hoja llamada “Notas” con los encabezados de las cuatro columnas y ninguna fila de datos.

¿Por qué sin datos?

En este primer paso lo importante es verificar que todo el pipeline funciona: el código Lisp se traduce a Python, el Python se ejecuta y se produce un archivo de salida (en este caso `.xlsx`). Una vez que esto funcione, podemos añadir contenido con tranquilidad.

1.4 Paso 2: Añadir los datos

Ahora que el sistema funciona, vamos a meter los datos en la tabla.

1.4.1 Los datos en *Common Lisp*

Los datos se guardan en una variable con `defparameter`. Cada fila es una lista con cuatro elementos (uno por columna). Las primeras tres filas se llaman de **prefijo**: no contienen datos como tal, sino que definen cómo se ve la tabla antes de los datos. En concreto: las dos primeras filas están en blanco (sirven como separadores visuales en el archivo de salida) y la tercera contiene los encabezados de columna. Las filas siguientes (a partir de la cuarta) son los datos propiamente dichos.

En la lista de datos, las tres primeras filas son de prefijo y el resto son datos:

Nombre	Nota 1	Nota 2	Promedio
Ana	85	90	—
Luis	70	75	—
Elena	92	88	—
Carlos	60	65	—
Sofía	78	82	—
Pedro	95	91	—

Listing 1.3: Datos de entrada.

```
1 (defparameter *DATOS*
2   '(((" " " " " " " ")
3     (" " " " " " " ")
4     ("Nombre" "Nota 1" "Nota 2" "Promedio")
5     ("Ana" "85" "90" " ")
6     ("Luis" "70" "75" " ")
7     ("Elena" "92" "88" " ")
8     ("Carlos" "60" "65" " ")
9     ("Sofia" "78" "82" " ")
10    ("Pedro" "95" "91" " ")))
```

Observe que la columna **Promedio** se deja con una cadena vacía () en los datos. En el código de la lista, cada par de comillas dobles representa ese valor vacío: es la manera de decirle al DSL que “esta celda no tiene valor fijo, la calculará automáticamente una expresión del lenguaje”. Nótese que en *Common Lisp* no es lo mismo que la lista vacía (): es un texto de longitud

cero, mientras que () es una lista sin elementos.

Formato de las celdas vacías

Las celdas que corresponden a columnas calculadas se marcan con (cadena vacía). Cada par en la lista de datos es un marcador de posición que el generador sustituye por la expresión correspondiente al generar el archivo de salida.

1.4.2 Modificar el libro para incluir los datos

Ahora modificamos el programa del paso 1 para que la tabla use los datos:

Listing 1.4: Paso 2: libro con datos.

```
1 (defparameter *DATOS*
2   '((" " " " " " " ")
3     (" " " " " " " ")
4     ("Nombre" "Nota 1" "Nota 2" "Promedio")
5     ("Ana" "85" "90" " ")
6     ("Luis" "70" "75" " ")
7     ("Elena" "92" "88" " ")
8     ("Carlos" "60" "65" " ")
9     ("Sofia" "78" "82" " ")
10    ("Pedro" "95" "91" " "))
11
12 (def-table tabla-notas
13   ((nombre "Nombre") (nota1 "Nota 1") (nota2 "Nota
14     2") (promedio "Promedio")))
15   :cell-height 1
16   :cell-width 1)
17
18 (libro tutorial-basico
19   :hojas (list
20     (hoja "Notas"
21       (tabla tabla-notas
22         :data *DATOS*)))
23
24   (xl-generate tutorial-basico "tutorial-basico.py")
25   (xl-run-generated "tutorial-basico.py"))
```

Ejecute de nuevo:

```
cd tutorial-basico/
```

```
sbcl --script tutorial-basico.lisp
```

Ahora el archivo de salida muestra los encabezados (**Nombre**, **Nota 1**, **Nota 2**, **Promedio**) y debajo las filas con los datos de cada persona. La columna **Promedio** sigue vacía porque aún no le hemos dicho cómo calcularla.

1.5 Paso 3: Columna calculada y resaltado

En este paso añadimos la expresión que calcula el promedio automáticamente y la regla que resalta en rojo los nombres de quienes superen la nota mínima.

1.5.1 Columna calculada

Queremos que los valores de la columna **promedio** se calculen automáticamente a partir de **nota1** y **nota2**:

$$\text{promedio} = \frac{\text{nota1} + \text{nota2}}{2}$$

En lenguaje natural diríamos: “para cada fila de la tabla, promedia los valores de las columnas **nota1** y **nota2**”. El DSL ofrece la forma **promedio** que hace exactamente eso: recibe una lista de columnas, genera la suma y la divide entre la cantidad de columnas automáticamente.

Listing 1.5: Tabla con columna calculada.

```
1 (def-table tabla-notas
2   ((nombre "Nombre") (nota1 "Nota 1") (nota2 "Nota
3     2") (promedio "Promedio")))
4   :cell-height 1
5   :cell-width 1
6   :computed
   ((promedio (promedio (col nota1) (col nota2)))))
```

Cada pieza significa:

- **(col nota1)** — toma el valor de la columna **nota1** en la fila actual (la fila que se está procesando en ese momento, distinta para cada alumno).
- **(promedio (col nota1) (col nota2))** — suma **nota1** y **nota2**, y divide entre 2 (porque recibe dos argumentos).

¿Cómo sabe promedio entre cuánto dividir?

`promedio` cuenta automáticamente los argumentos que recibe. Con `(promedio (col n1) (col n2))` genera $((n1+n2)/2)$; si recibe tres columnas genera $((n1+n2+n3)/3)$. No hace falta indicar el divisor.

Otras funciones disponibles

Además de `promedio`, el DSL ofrece otras funciones para columnas calculadas:

- `(add a b)` — suma dos valores.
- `(divide a b)` — divide a entre b.
- `(concat a b)` — concatena dos textos.

1.5.2 Resaltado condicional

Ahora queremos que los nombres de las personas cuyo promedio supere el mínimo aparezcan en rojo. En lenguaje natural: “si el promedio de esta fila es mayor que `nota-minima`, colorea el nombre”.

La opción `:render` de `def-table` acepta una forma `conditional-rendering` que especifica la condición y las columnas a colorear.

Listing 1.6: Tabla con columna calculada y resaltado.

```
1 (def-table tabla-notas
2   ((nombre "Nombre") (nota1 "Nota 1") (nota2 "Nota
3     2") (promedio "Promedio")))
4   :cell-height 1
5   :cell-width 1
6   :computed
7   ((promedio (promedio (col nota1) (col nota2))))
8   :render
9   (conditional-rendering
10    :condition (gt (col promedio) (param nota-minima))
11    :target-columns (nombre)))
```

Al leer la condición en voz alta:

- `gt` significa “mayor que” (*greater than*).
- `(col promedio)` es el promedio de la fila actual.
- `(param nota-minima)` es el valor fijo que definiremos.

- `:target-columns (nombre)` indica qué columna se colorea cuando la condición se cumple. Para colorear varias columnas se listan los nombres separados por espacio: `:target-columns (nombre promedio)`.

Múltiples columnas o condiciones

También puede resaltar más de una columna o usar varias condiciones en una sola regla. Por ejemplo:

- Para resaltar tanto nombres como promedios cuando el promedio es mayor que 80: `:target-columns (nombre promedio)`
- Para resaltar cuando el promedio es mayor que 80 y la nota 1 es mayor que 70: `(and (gt (col promedio) 80) (gt (col nota1) 70))`

El DSL permite combinar condiciones lógicas con `and`, `or` y `not` para crear reglas de resaltado complejas.

Más operadores de comparación

Además de `gt` (*greater than*), el DSL dispone de:

- `(lt a b)` — verdadero si a es menor que b.
- `(eq a b)` — verdadero si a es igual a b.
- `(and a b ...)` — verdadero si todos lo son.
- `(or a b ...)` — verdadero si al menos uno lo es.
- `(not a)` — verdadero si a es falso.

Con ellos puede construir condiciones compuestas, por ejemplo: “resaltar si el promedio es mayor que 80 y la nota 1 es menor que 90”.

1.5.3 El parámetro y la generación

Hemos usado `(param nota-minima)` en la condición, pero aún no hemos dicho de dónde sale ese valor. Los parámetros son valores fijos con nombre que se definen al instanciar la tabla, mediante la clave `:params`. Cada parámetro es un par `(nombre valor)`.

Listing 1.7: Libro completo con parámetros.

```

1 (libro tutorial-basico
2   :hojas (list
3     (hoja "Notas"
4       (tabla tabla-notas
5         :data *DATOS*
6         :params ((nota-minima 80))))))

```

```

7
8 (xl-generate tutorial-basico "tutorial-basico.py")
9 (xl-run-generated "tutorial-basico.py")

```

El nombre del archivo de salida lo define internamente el generador (por defecto `Archivo-Excel.xlsx`); no hace falta indicarlo en el DSL.

El concepto de parámetro

Un `param` es un valor fijo con nombre que se escribe una sola vez al instanciar la tabla y se referencia desde cualquier expresión de esa tabla con `(param nombre)`. Su alcance (*scope*) es la tabla donde se define: cada `tabla` recibe sus propios parámetros y no comparte los de otras tablas. Es la manera que tiene el DSL de representar constantes que puedan cambiar entre ejecuciones sin tocar las expresiones. En este ejemplo `(param nota-minima)` vale 80.

En la implementación concreta del generador a Excel, los parámetros se traducen a celdas fijas de la hoja (por ejemplo `$F$1`) para que las expresiones puedan referenciarlos con referencias absolutas. El backend de Python elige automáticamente una celda libre de la hoja y escribe el valor en ella, sin mezclarlo con las filas de datos de la tabla.

A continuación se muestra el contenido íntegro del archivo `tutorial-basico.lisp` con los tres pasos integrados:

1.6 El programa completo

Listing 1.8: `tutorial-basico.lisp` — programa completo.

```

1 (load (merge-pathnames "dsl-directo.lisp" *load-
2   truename*))
3 (defparameter *DATOS*
4   '((" " " " " ")
5     (" " " " " ")
6     ("Nombre" "Nota 1" "Nota 2" "Promedio")
7     ("Ana" "85" "90" " ")
8     ("Luis" "70" "75" " ")
9     ("Elena" "92" "88" " ")
10    ("Carlos" "60" "65" " ")
11    ("Sofia" "78" "82" " ")
12    ("Pedro" "95" "91" " ")))

```

```

13
14 (def-table tabla-notas
15   ((nombre "Nombre") (nota1 "Nota 1") (nota2 "Nota
16     2") (promedio "Promedio"))
17   :cell-height 1
18   :cell-width 1
19   :computed
20     ((promedio (promedio (col nota1) (col nota2))))
21   :render
22     (conditional-rendering
23       :condition (gt (col promedio) (param nota-
24         minima))
25       :target-columns (nombre)))
26
27 (libro tutorial-basico
28   :hojas (list
29     (hoja "Notas"
30       (tabla tabla-notas
31         :data *DATOS*
32         :params ((nota-minima 80)))))
33
34 (xl-generate tutorial-basico "tutorial-basico.py")
35 (xl-run-generated "tutorial-basico.py")

```

1.7 Ejecución

Para ejecutar el tutorial completo:

```

cd tutorial-basico/
sbcl --script tutorial-basico.lisp

```

Esto cargará el DSL, construirá el AST, generará `tutorial-basico.py` y lo ejecutará para crear `Archivo-Excel.xlsx`.

1.8 Resultado esperado

El archivo `Archivo-Excel.xlsx` contiene una hoja llamada “Notas” con la siguiente estructura:

Rango	Contenido
Filas 3–9, columnas A–D	Tabla con 6 personas (fila 3: encabezados)
Columna D, filas 4–9	Promedio calculado con expresión
Columna A, filas 4–9	Nombres en rojo si promedio > 80

Con los datos de ejemplo, los nombres de Ana, Elena, Sofía y Pedro aparecen resaltados en rojo porque su promedio supera 80. Luis y Carlos quedan con formato normal.

1.9 Resumen de formas utilizadas

Forma	Para qué sirve
<code>(def-table nombre (cols...) opts...)</code>	Define una tabla
<code>:computed ((col expr))</code>	Columna calculada con una expresión
<code>(conditional-rendering :condition :target-columns)</code>	Regla de resaltado condicional
<code>(promedio col1 col2 ...)</code>	Promedia las columnas indicadas
<code>(gt a b)</code>	Verdadero si a es mayor que b
<code>(col nombre)</code>	Valor de la columna en la fila actual
<code>(param nombre)</code>	Valor de un parámetro fijo
<code>:params ((nombre valor)...) (libro nombre ...)</code>	Define parámetros al instanciar la tabla
<code>(hoja nombre tablas ...)</code>	Define el libro completo
<code>(hoja nombre tablas ...)</code>	Define una hoja
<code>(tabla definicion :data :params)</code>	Instancia una tabla (definida con def-table) con datos y parámetros
<code>(xl-generate wb archivo)</code>	Genera el script Python
<code>(xl-run-generated archivo)</code>	Ejecuta el script y produce el archivo de salida (.xlsx)